

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Интеллектуальные методы анализа данных»
Тема: Первичный анализ и предобработка природного временного ряда

Студент гр. 3384

Горский К.Д.

Преподаватель

Мандрикова Б.С.

Санкт-Петербург

2026

Цель работы.

Повышение качества природных данных на примере наблюдений нейтронных мониторов.

Задание.

Вариант 5. Тикси (ТХВУ), 10-19 августа 2015 г.

1. Скачать данные из источника [<https://www.nmdb.eu/nest/>] за периоды и станции, указанные по вариантам в табл. (см. следующий слайд). Использовать данные с одномоментным временным разрешением.
2. Выполнить обнаружение пропусков и выбросов (3σ) в анализируемом временном ряде, посчитать их количество и долю от всех обрабатываемых данных.
3. С помощью функции `median` в MatLab вычислить медианное значение по данным без пропусков и выбросов.
4. Заменить пропуски и выбросы медианными значениями.
5. Изобразить графики исходного и полученного временных рядов.
6. Сделать соответствующие выводы, оформить отчет по работе.

Выполнение работы.

Работа выполнялась на языке Python с модулем `pandas` для работы с массивами данных и `matplotlib` для визуализации данных. Исходный код см. в приложении А.

Данные, скачанные в ASCII формате из упомянутого источника, соответствуют формату CSV (разделитель между значениями — символ «;»). Это позволяет считать данные из файла с помощью соответствующей функции в модуле `pandas`.

Для проверки корректности загрузки данных можно вывести первые 5 записей:

```
      datetime  RCORR_E
0 2015-08-10 00:00:00  111.067
1 2015-08-10 00:01:00  109.067
2 2015-08-10 00:02:00  110.000
3 2015-08-10 00:03:00  111.333
4 2015-08-10 00:04:00  108.000
```

Данные совпадают с содержимым файла.

С помощью встроенных в pandas функций можно вычислить среднее значение (mean) и стандартное отклонение (std) ряда. Диапазон 3σ вычисляется как (mean - $3 \cdot \text{std}$, mean + $3 \cdot \text{std}$). Полученные значения:

```
mean: 90.89303365150143
std: 8.335366813308141
range: (65.886933211577, 115.89913409142585)
```

Используя встроенные в pandas функции можно выделить все не входящие в диапазон 3σ записи и посчитать их количество. Полученное значение:

```
outliers: 124 (0.864%)
```

В скобках указана доля от общего количества записей.

Чтобы найти выбросы в данных, можно посчитать разницу во времени между всеми соседними записями и отобрать те пары, разница между которыми больше минуты. Полученные значения:

```
20      0 days 00:04:00
1490    0 days 00:04:00
1533    0 days 00:39:00
10093   0 days 00:04:00
```

У 20-й записи разница по времени с предыдущей записью — 4 минуты. Остальные строчки читаются аналогично. Суммарно пропусков:

```
n_missing: 47 (0.327%)
```

Суммарно пропусков и выбросов 171 (1.191% от всех данных).

Вычисленная медиана (если удалить выбросы):

```
median: 91.2
```

Далее необходимо заменить значения всех выбросов на вычисленную медиану, добавить пропуски (присвоить им медианное значение) и визуализировать исходные и обработанные данные.

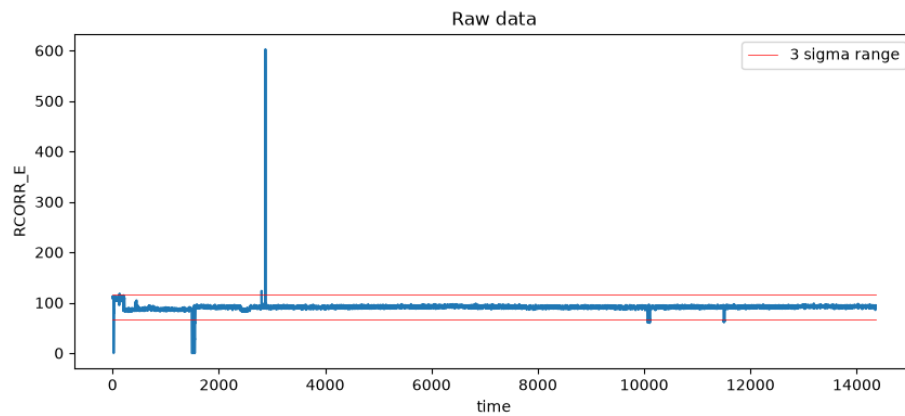


Рисунок 1 — Исходные данные.

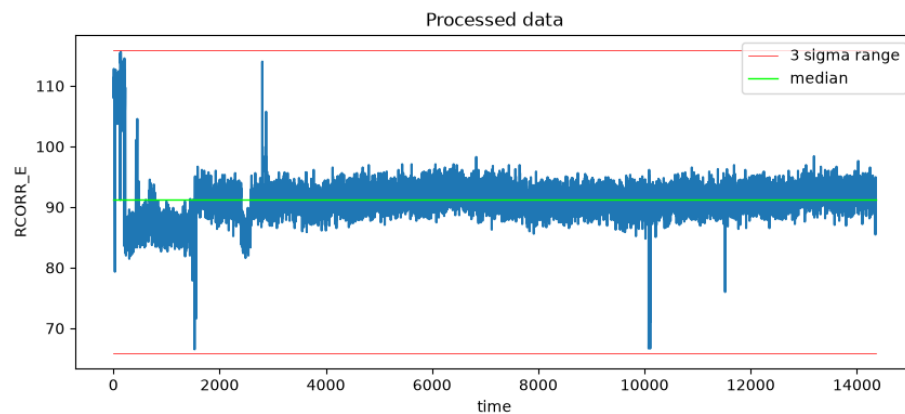


Рисунок 2 — Данные с заменой выбросов на медианное значение и добавлением пропусков.

На рис. 1 приведен график исходных данных. На рис. 2 приведен график данных, полученных из исходных данных путем заменой выбросов медианным значением и добавлением отсутствующих записей с присваиванием им медианного значения. Красными линиями отмечены границы 3σ диапазона, а зеленой линией отмечена медиана.

Выводы.

Повышено качество природных данных на примере наблюдений нейтронных мониторов. Определены (а после чего заменены на медианное значение) выбросы и пропуски в данных. Построены графики исходных данных и преобразованных. Судя по графикам, значения приобрели более адекватный диапазон. Всё ещё остались спорные наблюдения, находящиеся почти на границе 3σ диапазона, но исчезли очевидные выбросы (0.0 и 602.4).

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv('data.csv', skiprows=25, sep=';', names=['datetime',
'RCORR_E'])
df['datetime'] = pd.to_datetime(df['datetime'])
df['RCORR_E'] = pd.to_numeric(df['RCORR_E'])

print(df.head())

mean = df['RCORR_E'].mean()
std = df['RCORR_E'].std()

print(f"mean: {mean}")
print(f"std: {std}")

range = (float(mean - 3*std), float(mean + 3*std))
print(f"range: {range}")

n_total = len(df)

mask_outliers = (df['RCORR_E'] < range[0]) | (df['RCORR_E'] > range[1])
n_outliers = int(mask_outliers.sum())

print(f"total: {n_total}")
print(f"outliers: {n_outliers} ({n_outliers/14400*100:.3f}%)")

df_clean = df['RCORR_E'].drop(df[mask_outliers].index)
median = df_clean.median()
print(f"median: {median}")

df_new = df.copy()
df_new.loc[mask_outliers, 'RCORR_E'] = median

df_diff = df.datetime.diff()

print(df_diff[df_diff > pd.Timedelta('1min')])

n_missing = 14400 - len(df['RCORR_E'])
print(f"n_missing: {n_missing} ({n_missing/14400*100:.3f}%)")

print(f"n_missing+outliers: {n_missing+n_outliers}
({(n_missing+n_outliers)/n_total*100:.3f}%)")

df_new =
df.set_index('datetime').reindex(pd.date_range(df.datetime.min(),
df.datetime.max(), freq='1min')).reset_index()
df_new.columns = ['datetime', 'RCORR_E']
df_new['RCORR_E'] = df_new['RCORR_E'].fillna(median)

assert(len(df_new['RCORR_E']) == 14400)

minx = df.index.min()
```

```

maxx = df.index.max()

fig = plt.figure(figsize=(10, 4))
ax = fig.add_subplot(111)
ax.plot(df.index, df['RCORR_E'])
ax.set_xlabel('time')
ax.set_ylabel('RCORR_E')
ax.set_title('Raw data')
ax.plot((minx, maxx), (range[0], range[0]), 'r', linewidth=0.5, label="3
sigma range")
ax.plot((minx, maxx), (range[1], range[1]), 'r', linewidth=0.5)
ax.legend(loc=1)
fig.savefig("img/raw.png")

fig = plt.figure(figsize=(10, 4))
ax = fig.add_subplot(111)
ax.plot(df_new.index, df_new['RCORR_E'],)
ax.set_xlabel('time')
ax.set_ylabel('RCORR_E')
ax.set_title('Processed data')
ax.plot((minx, maxx), (range[0], range[0]), 'r', linewidth=0.5, label="3
sigma range")
ax.plot((minx, maxx), (range[1], range[1]), 'r', linewidth=0.5)
ax.plot((minx, maxx), (median, median), color='lime', linewidth=1,
label="median")
ax.legend(loc=1)
fig.savefig("img/processed.png")

print(df['RCORR_E'].min())
print(df['RCORR_E'].max())

```